

UNIT 2 · TECHNOLOGY FOR IoT

Programming for IoT

From code structure to digital & analog I/O on the ESP32

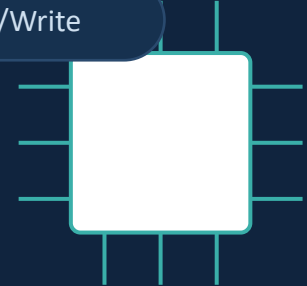
Code Structure

Directives

setup() / loop()

digitalRead/Write

analogRead/Write



What We'll Cover Today

01 Basic structure of an ESP32 / Arduino program

02 Preprocessor directives (#include, #define)

03 setup() and loop() — the two core functions

04 GPIO pins & pinMode()

05 digitalWrite() and digitalRead()

06 Analog signals, ADC, analogRead()

07 PWM & analogWrite()

Writing a Program for the ESP32

- A **program for a microcontroller** is called a sketch — written in a simplified C/C++.
- The **Arduino IDE** **compiles the sketch and uploads** it to the ESP32 over USB.
- The board **then runs that code independently, over and over**, reading sensors and controlling outputs.
- Every sketch follows the same predictable skeleton — once you know it, any new sketch is easy to read.

CODE → UPLOAD → RUN

1

Write sketch
in Arduino IDE

2

Upload via USB
(compiled to machine code)

3

ESP32 runs code
repeatedly

The Basic Structure of a Sketch

Every ESP32 / Arduino program has exactly three parts, always in this order.

```
// 1. DIRECTIVES
#include <Library.h>
#define LED_PIN 2

// 2. setup() -- runs ONCE
void setup() {
    pinMode(LED_PIN, OUTPUT);
}

// 3. loop() -- runs FOREVER
void loop() {
    digitalWrite(LED_PIN, HIGH);
}
```

1

Directives

Setup instructions for the compiler — libraries and constants.

2

setup()

Runs a single time when the board powers on or resets.

3

loop()

Runs continuously, forever, right after setup() finishes.

Preprocessor Directives

- Directives begin with # and are handled before compilation even starts.
- **#include** brings in a library's ready-made functions, e.g. WiFi or sensor drivers.
- **#define** creates a named constant that is swapped in everywhere it's used — easy to update once.
- Directives don't end with a semicolon, unlike normal C++ statements.

```
#include <WiFi.h>

#define LED_PIN 2
#define BAUD_RATE 115200
```

Tip: Think of #define as a global find-and-replace performed before your code even compiles.

void setup() — Runs Once

- Called a single time, right after the ESP32 boots up or is reset.
- Used for one-time preparation: configuring pin modes and starting communication.
- Typical jobs: pinMode(), Serial.begin(), connecting to Wi-Fi.
- Once setup() finishes, control passes automatically to loop().

```
void setup() {  
    pinMode(ledPin, OUTPUT);  
    pinMode(switchPin, INPUT);  
    Serial.begin(115200);  
}
```

void loop() — Runs Forever

- Executes immediately after setup(), then **repeats endlessly**.
- This is where the real behaviour of your device lives: reading sensors, reacting, updating outputs.
- Each pass-through loop() is sometimes called one "cycle" of the program.

```
void loop() {  
    digitalWrite(ledPin, HIGH);  
    delay(1000);  
    digitalWrite(ledPin, LOW);  
    delay(1000);  
}
```

setup() runs once → loop() repeats ∞ forever

GPIO Pins & pinMode()

- GPIO = General Purpose Input/Output — pins that can be configured as input or output in software.
- **pinMode**(pin, MODE) tells the ESP32 **how a pin will be used**, inside **setup()**.
- MODE is either OUTPUT (to drive a device) or INPUT (to read a signal).



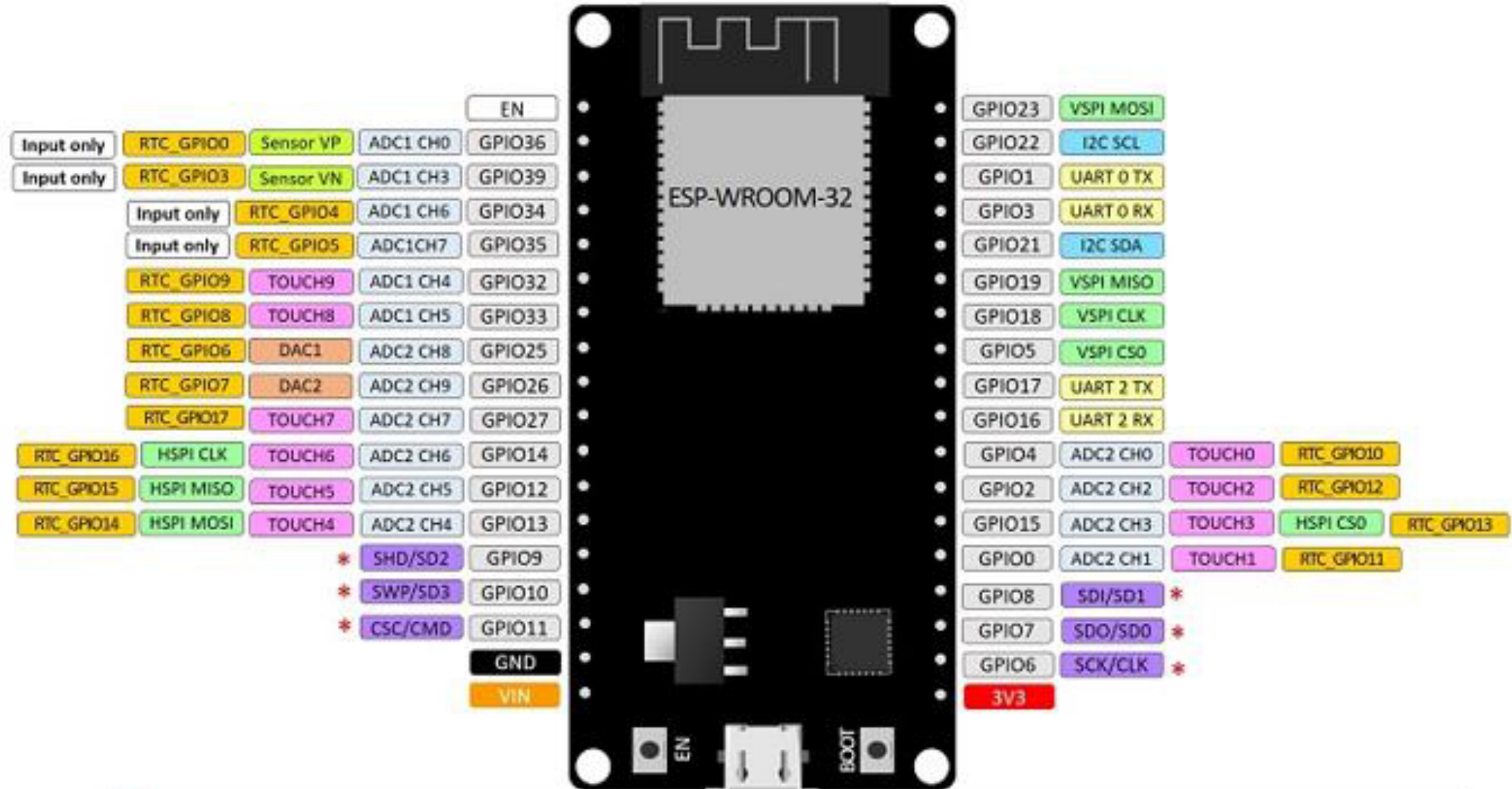
```
pinMode(ledPin, OUTPUT);  
pinMode(switchPin, INPUT);
```

ESP32 GPIO CONSIDERATIONS

- Logic level is 3.3 V — not 5 V tolerant
- GPIO 6–11 are reserved for internal flash; never use these
- GPIO 34–39 are input-only (no internal pull-ups)
- Internal pull-up resistors can be enabled in software

GPIO Pins

ESP32 DEVKIT V1 – DOIT version with 36 GPIOs



* Pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3 and CSC/CMD, namely, GPIO6 to GPIO11 are connected to the integrated SPI flash integrated on ESP-WROOM-32 and are not recommended for other uses.

digitalWrite() — Blinking an LED

OUTPUT

- **digitalWrite(pin, value)**
sets a pin HIGH (3.3 V, LED on) or LOW (0 V, LED off).
- The pin must first be set OUTPUT with pinMode() in setup().
- delay(ms) pauses the program for the given time in milliseconds.

LED anode → GPIO 2 (through a 220 Ω resistor)

LED cathode → GND

```
const int ledPin = 2;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  digitalWrite(ledPin, HIGH); // LED ON
  delay(1000);
  digitalWrite(ledPin, LOW);  // LED OFF
  delay(1000);
}
```

digitalRead() — Reading a Switch

INPUT

- digitalRead(pin) returns the current logic level of a pin: HIGH or LOW.
- The pin must be set INPUT with pinMode() in setup().
- Store the result in an int and act on it with an if / else check.

Switch terminal 1 → GPIO 4

Switch terminal 2 → GND

```
const int switchPin = 4;
const int ledPin = 2;

void setup() {
    pinMode(switchPin, INPUT_PULLUP);
    pinMode(ledPin, OUTPUT);
}

void loop() {
    int state = digitalRead(switchPin);

    if (state == LOW) {
        digitalWrite(ledPin, HIGH);
    } else {
        digitalWrite(ledPin, LOW);
    }
}
```

Analog Signals & the ADC

- Digital signals have only two states: HIGH or LOW.
- Analog signals vary continuously — e.g. a potentiometer, temperature or light sensor.
- The ESP32's built-in ADC (Analog-to-Digital Converter) turns a voltage into a number.
- 12-bit resolution → values from 0 to 4095, over 0 V to 3.3 V.

$$\text{Voltage} = (\text{ADC Value} / 4095) \times 3.3$$

ADC RANGE



0

4095

0 V

3.3 V

12-bit resolution: $2^{12} = 4096$ discrete steps

Recommended pin: potentiometer wiper → GPIO 34 (input-only ADC pin). Never exceed 3.3 V on this pin.

analogRead() — Reading a Potentiometer

ADC IN

- `analogRead(pin)` returns a number from 0–4095 representing the pin's voltage.
- Convert it to volts using the ADC formula from the previous slide.
- `Serial.begin()` and `Serial.print()` let you view live readings on the Serial Monitor.

Potentiometer wiper → GPIO 34

Outer terminals → 3.3 V and GND

```
const int potPin = 34;

void setup() {
    Serial.begin(115200);
}

void loop() {
    int adcValue = analogRead(potPin);
    float voltage = (adcValue/4095.0)*3.3;
    Serial.print("Voltage: ");
    Serial.println(voltage);
    delay(500);
}
```

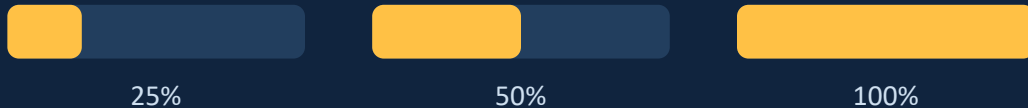
ANALOG OUTPUT

analogWrite() — PWM Output

PWM OUT

- The ESP32 has no true analog output — analogWrite() simulates it using PWM (Pulse Width Modulation).
- It rapidly switches a pin HIGH/LOW; the value 0–255 sets how much of the time it stays HIGH.
- Used for LED brightness, motor speed, and buzzer tone control.

DUTY CYCLE = BRIGHTNESS



```
const int ledPin = 5;    // PWM-capable
                           pin

void setup() {
    pinMode(ledPin, OUTPUT);
}

void loop() {
    analogWrite(ledPin, 64);    // ~25%
    delay(1000);
    analogWrite(ledPin, 128);  // ~50%
    delay(1000);
    analogWrite(ledPin, 255);  // 100%
    delay(1000);
}
```

Your ESP32 Function Toolkit

pinMode(pin, mode)

Configure a pin as INPUT or OUTPUT

Setup

digitalWrite(pin, val)

Set a pin HIGH or LOW

Digital Out

digitalRead(pin)

Read HIGH or LOW from a pin

Digital In

analogRead(pin)

Read a 0–4095 value from an ADC pin

Analog In

analogWrite(pin, val)

Output a PWM signal, 0–255

Analog Out

Defining Your Own Function

- A function is a named, reusable block of code — `setup()` and `loop()` are functions too.
- `returnType` is what the function gives back — use `void` if it returns nothing.
- Parameters (in the parentheses) are inputs the function receives.
- Call the function by name wherever you need it, instead of repeating code.

```
// returnType functionName(parameters)
bool isPressed(int pin) {
    return digitalRead(pin) == LOW;
}

void blinkLed(int pin, int times) {
    for (int i = 0; i < times; i++) {
        digitalWrite(pin, HIGH);
        delay(200);
        digitalWrite(pin, LOW);
        delay(200);
    }
}

void loop() {
    if (isPressed(4)) blinkLed(2, 3);
}
```

Macro vs Function — What's the Difference?

MACRO (#define)

```
#define SQUARE(x) ((x)*(x))
```

- Replaced by the preprocessor before compiling — a pure text substitution.
- No type checking — can cause subtle bugs.
- No debugging step-through; it never really "exists" as code.

FUNCTION

```
int square(int x) { return x * x; }
```

- Compiled as real code — has proper type checking.
- Easier to debug (breakpoints, step-through).
- Preferred for anything beyond a trivial one-line swap.